A Physics-Informed Ocean Emulator Trained on CESM–LENS: Architecture, Training Data, and Modes of Variability

Aneesh Subramanian et al.

June 19, 2026

Contents

1 Introduction and motivation

An *ocean emulator* is a machine-learning model that has learned to integrate the ocean equations of motion from data. Given the current three-dimensional state of the ocean and the surface boundary conditions (heat fluxes, wind stress, freshwater input), the emulator predicts how the ocean evolves over the next time step—mimicking what an ocean general circulation model (OGCM) does through explicit numerical integration, but at a fraction of the computational cost.

The parent model here is POP2 (Parallel Ocean Program version 2), the ocean component of NCAR's Community Earth System Model. POP2 is run as part of the CESM Large Ensemble (CESM–LENS), a collection of over 40 historical and future-scenario simulations that differ only in their initial atmospheric perturbation. This ensemble provides a uniquely large corpus of physically-consistent, high-quality ocean trajectories—ideal for supervised machine-learning because the training labels (future ocean states) are simply the subsequent months of the same simulation.

This document is written for a first-year graduate student who is comfortable with basic differential equations and Python but may be new to deep learning or ocean modeling. It describes the full pipeline from raw data on disk to a trained model capable of multi-year free-running forecasts, and evaluates how well the model reproduces the leading modes of ocean variability.

2 The parent model: POP2 and the CESM Large Ensemble

2.1 POP2 primitive equations

POP2 solves the hydrostatic, Boussinesq primitive equations on a staggered finite-volume grid. Let $\mathbf{u} = (u, v)$ be the horizontal velocity, w the vertical velocity, T potential temperature, S salinity, ρ density, p pressure, and f the Coriolis parameter. The governing equations are:

Horizontal momentum

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} + w\frac{\partial \mathbf{u}}{\partial z} + f\hat{k} \times \mathbf{u} = -\frac{1}{\rho_0}\nabla_h p + \nabla \cdot (A_h \nabla \mathbf{u}) + \frac{\partial}{\partial z}\left(A_v \frac{\partial \mathbf{u}}{\partial z}\right) \tag{1}$$

Hydrostatic balance

$$\frac{\partial p}{\partial z} = -\rho\, g \tag{2}$$

Continuity (Boussinesq incompressibility)

$$\nabla_h \cdot \mathbf{u} + \frac{\partial w}{\partial z} = 0 \tag{3}$$

Tracer transport (for $C \in \{T, S\}$)

$$\frac{\partial C}{\partial t} + \nabla_h \cdot (\mathbf{u}C) + \frac{\partial (wC)}{\partial z} = \nabla \cdot (\kappa_h \nabla C) + \frac{\partial}{\partial z}\left(\kappa_v \frac{\partial C}{\partial z}\right) + Q_C \tag{4}$$

Equation of state

$$\rho = \rho(T, S, p) \quad \text{(MWJF 2003, Section 2.2)} \tag{5}$$

Surface boundary conditions (the forcing) The ocean surface is driven by:

$$A_v \frac{\partial \mathbf{u}}{\partial z}\bigg|_{z=0} = \boldsymbol{\tau}/\rho_0 \quad \text{(wind stress; TAUX, TAUY)} \tag{6}$$

$$\kappa_v \frac{\partial T}{\partial z}\bigg|_{z=0} = Q_{\text{heat}}/(\rho_0 c_p) \quad \text{(SHF, SHF\_QSW)} \tag{7}$$

$$\kappa_v \frac{\partial S}{\partial z}\bigg|_{z=0} = F_{\text{salt}} \quad \text{(virtual salt flux from SFWF)} \tag{8}$$

These five surface flux fields are exactly what the emulator receives as *forcing inputs* each time step; they are what "closes" the conservation budgets (Section 7).

2.2 Equation of state: MWJF 2003

POP uses the McDougall–Wright–Jackett–Feistel (2003) equation of state—a 25-term rational polynomial that expresses in-situ density as:

$$\rho(T, S, p) = \frac{P_1(T, S, p)}{P_2(T, S, p)} \tag{9}$$

where P_1 (numerator) and P_2 (denominator) are polynomials in potential temperature T [°C], salinity S [psu], and pressure p [dbar]. P_2 includes a $S^{3/2}$ term to capture the nonlinear effect of salinity on compressibility. The MWJF EOS matches EOS-80 to better than $0.01\,\text{kg}\,\text{m}^{-3}$ at the surface—for example, $\rho(35\,\text{psu}, 25°\text{C}, 0) = 1023.34\,\text{kg}\,\text{m}^{-3}$.

The emulator implements this *exact same polynomial* in `src/pop_emulator/physics.py` (`mwjf_density`), so density computed from the emulator's predicted (T, S) fields is physically consistent with the parent model. One numerical subtlety: land/masked cells carry $S = 0$ in the data, and $\partial\sqrt{S}/\partial S \to \infty$ at $S = 0$ would produce NaN gradients during back-propagation. The fix is to clamp salinity at a small floor value ($S_{\min} = 0.01$ psu) before the square root; this has no physical consequence because masked cells are excluded from every loss term.

2.3 CESM Large Ensemble

CESM–LENS comprises three main experiment families (Table 1). The emulator is trained on the *historical* experiment members because: (1) the full set of 3-D monthly ocean fields is available; (2) the climate forcing is realistic (20th-century greenhouse gases, aerosols, solar variability); and (3) 19 independent members provides hundreds of training years.

Table 1: CESM–LENS experiment families available on GLADE.

Tag	Experiment	Period	Members used
`B20TRC5CNBDRD`	Historical	1850–2005	19 train, 1 val, 2 test
`BRCP85C5CNBDRD`	RCP8.5 future	2006–2100	(future expansion)
`B1850C5CN`	Pre-industrial control	multi-century	(not yet used)

Each member is an independent integration: it shares the same model code and external forcing as every other member, but was initialized from a slightly different atmospheric state (a so-called "micro perturbation"). This means the ensemble samples the natural internal variability of the climate system — each member follows a different realization of El Niño, the PDO, etc. — while the underlying physical laws are identical. For machine learning this is ideal: the model sees many different ocean trajectories from the same dynamics, reducing the risk of learning spurious correlations from a single trajectory.

Member 001 is **held out** as the validation set and never seen during training; members 021 and 022 are the test set. The train/val/test split is by member, not by time, so there is no look-ahead contamination.

3 Training data: variables, grid, and file layout

3.1 Grid: gx1v6 displaced-pole

POP uses a "nominal 1°" displaced-pole grid called `gx1v6`. Unlike a regular latitude–longitude grid, the north pole of the model grid is *displaced* into Canada, so that the grid is smooth and orthogonal over the Arctic Ocean (which is the most challenging region for numerical stability in a lat–lon grid). The grid dimensions are:

$$(n_{\text{lat}},\, n_{\text{lon}},\, n_z) = (384,\, 320,\, 60)$$

so a single 3-D field is $384 \times 320 \times 60 = 7{,}372{,}800$ grid cells. The 60 vertical levels span from 5 m at the surface to 5375 m at the bottom, with thin layers near the surface (where variability is greatest) and thick layers in the abyss.

B-grid staggering. POP uses a *B-grid* (Arakawa B-grid): tracers (T, S) are stored at the centre of each cell ("T-point"), while horizontal velocities (u, v) are stored at the north-east corner of each cell ("U-point"). This means the divergence of the velocity field—needed to reconstruct the vertical velocity—is evaluated on the T-grid using the four surrounding U-points. The sea-surface height (SSH) lives at T-points because it is a tracer-type variable in POP's free-surface formulation.

Topography: KMT and partial bottom cells. The ocean bottom is represented by the integer field `KMT(j,i)`, the index of the deepest active level in each (j, i) column. A cell is "ocean" if its level index $k < \texttt{KMT}(j, i)$, otherwise it is "land" (or rock below the sea floor). The 3-D ocean mask $\texttt{mask3d}(k, j, i)$ encodes this and is applied after every model output so that land cells always carry zero signal.

3.2 Variables

3.2.1 Prognostic state (what the emulator integrates)

Table 2: State variables predicted by the emulator at each time step.

POP name	Physical quantity	Grid	Shape per month	Units
`TEMP`	Potential temperature	3-D T-grid	$(60, 384, 320)$	°C
`SALT`	Salinity	3-D T-grid	$(60, 384, 320)$	$\mathrm{g\,kg^{-1}}$ (psu)
`UVEL`	Grid-x velocity	3-D U-grid	$(60, 384, 320)$	$\mathrm{cm\,s^{-1}}$
`VVEL`	Grid-y velocity	3-D U-grid	$(60, 384, 320)$	$\mathrm{cm\,s^{-1}}$
`SSH`	Sea-surface height	2-D T-grid	$(384, 320)$	cm

Vertical velocity (`WVEL`) is *not* predicted by the network. Instead it is *diagnosed* from the continuity equation ($w(k) = w(k+1) + dz_k\,(\nabla_h \cdot \mathbf{u})_k$, integrated upward from the rigid floor). This construction guarantees discrete incompressibility to machine precision, regardless of any error the network makes in (u, v).

3.2.2 Forcing inputs (what drives the emulator at each step)

Table 3: Surface forcing fields used as network inputs at each time step.

POP name	Physical meaning	Units	Role
`SHF`	Net surface heat flux	$\mathrm{W\,m^{-2}}$	Closes heat budget
`SHF_QSW`	Shortwave (penetrative solar)	$\mathrm{W\,m^{-2}}$	Separated from SHF
`SFWF`	Virtual salt / freshwater flux	$\mathrm{kg\,m^{-2}\,s^{-1}}$	Closes salt budget
`TAUX`	Zonal wind stress	$\mathrm{N\,m^{-2}}$	Momentum surface BC
`TAUY`	Meridional wind stress	$\mathrm{N\,m^{-2}}$	Momentum surface BC

3.2.3 Static grid channels (constant in time)

Seven time-invariant fields are concatenated to the input at every step so the network can represent geostrophic dynamics and respect topography:

1. Coriolis parameter f (normalized), enables geostrophic balance
2. 2-D ocean mask (tells the network where the ocean is)
3. Normalized column depth (from `KMT`)
4. $\sin(\phi)$, $\cos(\phi)$: latitude encoding
5. $\sin(\lambda)$, $\cos(\lambda)$: longitude encoding

These channels are generated once from `data/grid_gx1v6.nc` and broadcast to every batch element.

3.3 File layout and size on disk

The data live on GLADE under:

```
/glade/campaign/collections/gdex/data/d651027/cesmLE/
  CESM-CAM5-BGC-LE/ocn/proc/tseries/monthly/
    TEMP/ SALT/ UVEL/ VVEL/ SSH/
    SHF/ SHF_QSW/ SFWF/ TAUX/ TAUY/
```

Each subdirectory holds one NetCDF file per member (or per decade, split across multiple files for long runs). A single member's TEMP field for 1850–2005 contains 1,872 monthly snapshots of $60 \times 384 \times 320$ float32 values, totalling ≈ 23 GB. With 22 members and 10 variables, the full corpus is ≈ 5 TB. **Loading this into RAM is impossible**—the data pipeline must stream lazily from disk.

4 Data loading pipeline

4.1 Lazy loading with xarray and dask

Raw data is accessed through `xarray.open_dataset` with `chunks={"time": 1}`. This tells xarray to use dask for lazy evaluation: rather than reading the entire file into RAM, dask builds a task graph that reads only the one time slice actually needed. Each `isel(time=t)` call materializes a single $60 \times 384 \times 320$ array (≈ 28 MB for float32), which is small enough to hold comfortably in RAM. Variables are opened lazily once and cached in a Python dict, so the file is not re-opened on every call.

Listing 1: Lazy opening of a variable.

```
# xarray opens the file header only; no data read yet.
da = xr.open_dataset(file, decode_times=False, chunks={"time": 1})[var]
# Only now is one time slice read from disk:
arr = da.isel(time=t).values # (Z, J, I) numpy array
```

Land/fill values in POP are marked with a sentinel $\approx 10^{30}$. The `_clean` function replaces any non-finite value or any value $> 10^{30}$ with zero, consistent with the ocean mask.

4.2 The sample index

Training proceeds by sampling *(member, t_0)* pairs. For each member and each valid time t_0, a sample consists of:

- H consecutive input states ending at t_0 (the *history*; $H = 2$ in the current configuration)
- R target states at $t_0 + 1$, $t_0 + 2, \ldots, t_0 + R$ (the *rollout* targets; R follows the curriculum)
- R forcing fields at t_0, $t_0 + 1, \ldots, t_0 + R - 1$

The dataset class (`PopLENSDataset`) pre-builds the full index list at construction time—checking that there are enough months on both sides of every t_0 to form a complete sample. Training starts at month `t_start=360` (skipping the first 30 years of spin-up in the 1850-start members).

For 19 training members each with $\approx$ 1,500 usable months and rollout $R = 8$, the dataset has $\approx 19 \times 1{,}492 = 28{,}348$ samples per epoch.

4.3 The PyTorch DataLoader

`make_loader` wraps `PopLENSDataset` in a standard PyTorch `DataLoader`:

```
DataLoader(
    ds,
    batch_size=1, # one global-ocean field per sample (memory-limited)
    shuffle=True, # randomize (member, t0) order each epoch
    num_workers=4, # 4 CPU worker processes prefetch in parallel
    collate_fn=collate, # stack list-of-dicts into batched dict-of-tensors
    pin_memory=True, # pinned (page-locked) RAM for zero-copy GPU transfer
    drop_last=True, # discard last partial batch during training
    persistent_workers=True, # keep workers alive between batches
)
```

CPU workers and prefetching. The `num_workers=4` setting spawns 4 separate CPU processes. Each worker independently selects a (member, t_0) pair, calls `xarray` to read the relevant slices from disk, performs the land-mask clean, converts to PyTorch tensors, and places the result in a shared-memory queue. The main training process fetches pre-built batches from the queue, so disk I/O and GPU computation overlap: while the GPU is doing a forward pass on batch i, the CPU workers are already reading and preprocessing batch $i+1$. This pipelining is crucial because loading a batch from disk can take 0.5 s or more, while a GPU forward pass takes ≈ 0.1 s.

Pinned memory. With `pin_memory=True`, the DataLoader places tensors in *page-locked* (pinned) host RAM. This enables the CUDA driver to transfer data from CPU to GPU using DMA (direct memory access) without an intermediate copy, reducing the CPU-to-GPU transfer latency by $\sim 2\times$. Pinned memory is used automatically by PyTorch's `.to(device)` call.

Batch size. The batch size is 1 (one global-ocean snapshot per gradient update). A single sample contains $2 \times (4 \times 60 + 1) = 482$ input state channels at (384×320) spatial resolution, requiring ≈ 1.4 GB of GPU memory for the activations in a single forward pass. With a 4-encoder U-Net at width 96, the total peak GPU memory per training step (forward + backward + optimizer) is ≈ 30 GB on the A100 at 8-step rollout. The A100 has 80 GB of HBM2e, so the model fits comfortably at this batch size.

Gradient accumulation (`grad_accum=4`) simulates a logical batch size of 4 by accumulating gradients across 4 sequential forward/backward passes before each optimizer step.

5 State packing and normalization

5.1 StatePacker: dict-of-fields to channel tensor

The neural network operates on a single 4-D tensor of shape (B, C, J, I) where B is the batch size, C is the total number of channels, and $(J, I) = (384, 320)$ is the horizontal grid. The `StatePacker` class converts between the dict-of-fields representation (one tensor per variable) and this packed channel tensor:

$$C = n_{3D} \times n_z + n_{2D} = 4 \times 60 + 1 = 241 \text{ channels per state snapshot} \tag{10}$$

Channel order: TEMP levels 0–59, SALT levels 0–59, UVEL levels 0–59, VVEL levels 0–59, SSH (1 channel). With history $H = 2$, the input state block is $2 \times 241 = 482$ channels (oldest state first, newest last).

5.2 Per-level standardization

Ocean fields vary enormously with depth: surface temperature varies by tens of degrees seasonally, while the deep ocean is nearly constant at $\approx 2\,^{\circ}$C. Using a single global mean/std would allow the surface to dominate the loss, and the optimizer would ignore the deep ocean entirely.

Instead, each level gets its own mean and standard deviation, computed from the training members by `scripts/compute_stats.py` and stored in `data/stats.nc`. Normalization then maps each channel to zero mean and unit variance:

$$\tilde{x}_c = \frac{x_c - \mu_c}{\sigma_c} \tag{11}$$

where c indexes the packed channel. The same statistics are used for all time steps and members (global statistics over the training set), computed excluding land cells.

5.3 Tendency-weighted loss

A subtle but critical issue: if the loss is the MSE of the *normalized field*—i.e. $\|\tilde{x}^{t+1}_{\text{pred}} - \tilde{x}^{t+1}_{\text{true}}\|^2$—then predicting *no change* ("persistence") gives a loss equal to the variance of the field in normalized space. For deep ocean levels, where month-to-month changes are tiny relative to the field magnitude, persistence is nearly a perfect predictor in normalized units, and the optimizer collapses to it. This was observed in Run 1.

The fix is to re-weight each channel by $(\sigma_{\text{field}}/\sigma_{\text{tendency}})^2$, where σ_{tendency} is the standard deviation of the month-to-month change. In tendency-weighted space, persistence (zero tendency) gives a loss of $(\sigma_{\text{field}}/\sigma_{\text{tendency}})^2 \sim O(1)$ per channel—the optimizer *must* learn the real dynamics.

$$L_{\text{data}} = \sum_c w_c \|\tilde{x}^{\text{pred}}_c - \tilde{x}^{\text{true}}_c\|^2 \quad \text{where } w_c = \left(\frac{\sigma_{\text{field},c}}{\sigma_{\text{tend},c}}\right)^2 \tag{12}$$

Weights are clipped at 10^4 to prevent any single channel from dominating.

6 Model architecture

6.1 Overview

The network predicts a *tendency*: the update to add to the current state. This mirrors POP's own time-stepping and provides a natural inductive bias (the tendency is small relative to the field, so the network can start near zero).

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \Delta t \cdot F_\theta(\mathbf{x}(t), \mathbf{b}(t), \mathbf{g}) \tag{13}$$

where F_θ is the neural network, $\mathbf{b}(t)$ is the surface forcing, and $\mathbf{g}$ denotes the static grid fields. The output of F_θ is in normalized units, and the land mask is re-applied after the residual update so that land cells are always exactly zero.

6.2 Spherical U-Net

The backbone is a **U-Net** (Ronneberger et al. 2015), a convolutional encoder-decoder architecture that processes the global ocean field at multiple spatial scales simultaneously. A standard U-Net would not work here because:

- Longitude is *periodic*: the Atlantic and Pacific are connected at the dateline. Standard zero-padding would create a spurious boundary.
- Latitude is *non-periodic*: the equator and poles are distinct. The Arctic (the POP tripole fold) is approximated by replicate padding.

The solution is `OceanConv2d`, a drop-in replacement for `nn.Conv2d` that always pads with circular longitude and replicate latitude before each convolution:

```
def ocean_pad(x, pad=1):
    x = torch.cat([x[..., -pad:], x, x[..., :pad]], dim=-1) # lon: wrap
    x = F.pad(x, (0, 0, pad, pad), mode="replicate") # lat: replicate
    return x
```

Architecture summary. Table 4 gives the key hyperparameters. The network consists of:

1. A **stem convolution** projecting the input channels to width 96.
2. Four **encoder stages**: each runs 2 residual blocks (GroupNorm + SiLU activation + 3×3 OceanConv2d, repeated twice with a skip connection), then a stride-2 OceanConv2d that doubles channels and halves the spatial resolution. After 4 stages the spatial resolution is $\lfloor 384/16 \rfloor \times \lfloor 320/16 \rfloor = 24 \times 20$ at $96 \times 2^4 = 1536$ channels.
3. A **bottleneck**: two residual blocks with a FiLM layer in between (Section 6.3).
4. Four **decoder stages**: bilinear upsampling followed by a skip connection from the matching encoder stage (concatenated on the channel axis), then 2 residual blocks. The decoder mirrors the encoder in reverse.
5. An **output head**: GroupNorm + SiLU + 3×3 OceanConv2d projecting to 241 channels (one per state variable/level). The output convolution is initialized to all zeros, so the model starts by predicting zero tendency (i.e. identity mapping)—this greatly stabilizes early training.

Table 4: Model architecture and parameter count.

Hyperparameter	Value
Architecture	Spherical U-Net (grid-aware padding)
Base channel width	96
Encoder/decoder depth	4 stages
Residual blocks per stage	2
Normalization	GroupNorm (groups=8)
Activation	SiLU (Sigmoid Linear Unit)
Input state channels	$2 \times 241 = 482$ (history $H = 2$)
Forcing channels	5
Static channels	7
Total input channels	$482 + 5 + 7 = 494$
Output channels	241
FiLM conditioning dim	$2 + 5 = 7$ (month embedding + forcing means)
Total parameters	≈ 189 million
Output initialization	zeros (identity start)

6.3 FiLM conditioning

A critical design choice is how to inform the network about the *current month* (so it can represent the seasonal cycle of mixed-layer depth, sea ice, etc.) and the global magnitude of the surface forcing (so it knows how energetic the current step is).

This is done with **Feature-wise Linear Modulation (FiLM)**: a small multi-layer perceptron maps the conditioning vector to per-channel scale (γ) and shift (β) parameters, which modulate the bottleneck feature maps:

$$\text{FiLM}(h, \mathbf{c}) = h \cdot (1 + \gamma(\mathbf{c})) + \beta(\mathbf{c}) \tag{14}$$

where h is the bottleneck feature tensor (B, C, J, I) and γ, β are (B, C) vectors broadcast over the spatial dimensions. This allows the same network weights to produce seasonally-varying outputs with minimal parameter overhead.

The conditioning vector $\mathbf{c}$ is:

$$\mathbf{c} = \left[\sin\left(\frac{2\pi m}{12}\right), \cos\left(\frac{2\pi m}{12}\right), \overline{f}_1, \ldots, \overline{f}_5\right] \tag{15}$$

where $m \in \{0, \ldots, 11\}$ is the month-of-year and $\overline{f}_k$ is the spatial mean of normalized forcing variable k (capturing the global strength of each forcing). The cyclic $(\sin, \cos)$ encoding ensures the network sees December and January as close in season-space.

6.4 2-state input history

A key architectural choice inspired by the Samudra ocean emulator (Dheeshjith et al. 2025) is to condition the network on the *two most recent* states rather than just the current state. Providing $\mathbf{x}(t-1)$ alongside $\mathbf{x}(t)$ gives the network access to the local tendency $\mathbf{x}(t) - \mathbf{x}(t-1)$, effectively encoding momentum and acceleration without a separate tendency variable. This was shown by Samudra to enable century-scale stable rollouts of a global ocean emulator.

The two states are stacked on the channel dimension (oldest first):

$$\text{input state block} = [\mathbf{x}(t-1),\, \mathbf{x}(t)] \quad \text{shape: } (B,\, 2 \times 241,\, 384,\, 320) \tag{16}$$

The residual update is always applied to the most recent state $\mathbf{x}(t)$:

$$\mathbf{x}(t+1) = \mathbf{x}(t) + F_\theta([\mathbf{x}(t-1), \mathbf{x}(t)],\, \mathbf{b}(t)) \tag{17}$$

7 Physics constraints in the loss function

The network is trained not just to minimize prediction error but also to satisfy the physical constraints of the POP2 equations. These constraints are incorporated as *soft penalty terms* added to the data loss. The full composite loss is:

$$L = L_{\text{data}} + \lambda_{\text{cont}}\, L_{\text{cont}} + \lambda_{\text{baro}}\, L_{\text{baro}} + \lambda_{\text{heat}}\, L_{\text{heat}} + \lambda_{\text{salt}}\, L_{\text{salt}} + \lambda_{\text{stab}}\, L_{\text{stab}} \tag{18}$$

Table 5: Physics loss weights by run. Run-4 heat and salt weights are 0 because those budgets are closed exactly by the conservation projection (Section 7.4) rather than by a soft penalty.

Loss term	Run-3 λ	Run-4 λ	Physical constraint enforced
L_{data}	1.0	1.0	Prediction accuracy (MSE vs POP targets)
L_{cont}	0.1	0.1	Rigid-lid: surface vertical velocity ≈ 0
L_{baro}	0.1	0.1	Volume conservation: barotropic divergence ≈ 0
L_{heat}	0.02	0.0	Heat-content change $\approx$ SHF integral (soft)
L_{salt}	0.02	0.0	Salt-content change $\approx$ SFWF integral (soft)
L_{stab}	0.02	0.02	No spurious gravitational instability

All physics weights are ramped from 0 to their target value over 2000 "warmup steps" at the start of training. This prevents the physics constraints from fighting the data loss during the early phase when the network is still learning the basic mapping. In run 4, heat and salt weights are 0 because the conservation projection (Section 7.4) closes those budgets exactly at every step—the soft penalty would be redundant.

7.1 Continuity and barotropic divergence

The rigid-lid condition (POP does not have a free surface at the barotropic level in this configuration) requires that the vertically integrated horizontal divergence vanish:

$$L_{\text{baro}} = \left\| \sum_k dz_k \, (\nabla_h \cdot \mathbf{u})_k \right\|^2_{\text{ocean}} \tag{19}$$

The surface continuity penalty enforces the rigid-lid kinematic condition:

$$L_{\text{cont}} = \|w_{\text{diag}}(k = 0)\|^2_{\text{ocean}} \tag{20}$$

where w_{diag} is the vertical velocity diagnosed from the continuity equation (Eq. 3). If the predicted (u, v) field is perfectly non-divergent, then w at the surface is exactly zero.

7.2 Heat and salt budget conservation

The global ocean heat content is $H = \int \rho_0 \, c_p \, T \, dV$. Over one model time step, this should change by exactly the surface heat flux integrated over the ocean surface:

$$\Delta H_{\text{pred}} \approx \Delta t \int_{\text{surface}} \text{SHF} \, dA \tag{21}$$

The corresponding loss uses a *bounded relative closure error*:

$$L_{\text{heat}} = \left(\frac{\Delta H_{\text{pred}} - \Delta H_{\text{flux}}}{|\Delta H_{\text{pred}}| + |\Delta H_{\text{flux}}| + \epsilon_H} \right)^2 \tag{22}$$

The denominator prevents division-by-zero when both the predicted change and the flux-implied change are near zero (which happens at equilibrium), while keeping the loss dimensionless and $\mathcal{O}(1)$ for real errors. The same form is used for salt.

Why this form matters. An earlier version used $L_{\text{heat}} = (\Delta H_{\text{pred}} - \Delta H_{\text{flux}})^2/\text{const}^2$ with a hand-tuned scale. When the flux happened to be near zero in a batch, the scale was tiny and the loss term exploded to $\sim 10^3$, destabilizing training. The bounded relative error (Eq. 22) is robust to this because the normalization grows with the violation rather than being fixed.

7.3 Static stability penalty

A gravitationally unstable water column (denser water over lighter water) is convectively adjusted by POP's mixing scheme. The emulator cannot run a convective adjustment, so it is discouraged from inventing instability beyond what POP already has in the target field:

$$L_{\text{stab}} = \Big\langle \text{ReLU}\Big(\rho_k(T^{\text{pred}}, S^{\text{pred}}) - \rho_{k+1}(T^{\text{pred}}, S^{\text{pred}})\Big) - \text{ReLU}\big(\rho_k(T^{\text{true}}, S^{\text{true}}) - \rho_{k+1}(T^{\text{true}}, S^{\text{true}})\big) \Big\rangle \tag{23}$$

where both density profiles are computed at a common interface pressure (using the MWJF EOS) so the comparison is between potential densities.

7.4 Conservation projection (Run 4)

Runs 1–3 used the soft heat and salt budget losses (Eq. 22) to *encourage* conservation. Even with small penalties, a soft constraint allows residual per-step errors that accumulate secularly over decades—exactly the drift identified as the cause of the 307-month stability ceiling in run 3.

Run 4 replaces the soft penalties with a **hard conservation projection**: after every model step, the heat and salt budget residuals are computed and removed by applying a spatially uniform correction $(\delta T, \delta S)$ to every ocean cell:

$$\delta T = -\frac{\Delta H_{\mathrm{pred}} - \Delta t \int_{\mathrm{surface}} \mathrm{SHF}\, dA}{\rho_0\, c_p\, V_{\mathrm{ocean}}} \tag{24}$$

$$\delta S = -\frac{\Delta S_{\mathrm{pred}} - \Delta t \int_{\mathrm{surface}} \mathrm{SFWF} \cdot S_{\mathrm{ref}}\, dA}{\rho_0\, V_{\mathrm{ocean}}} \tag{25}$$

where $V_{\mathrm{ocean}} = \int_{\mathrm{ocean}} dV$ is the total ocean volume, $\rho_0 = 1026\,\mathrm{kg\,m^{-3}}$ is the reference density, and $c_p = 3996\,\mathrm{J\,kg^{-1}\,K^{-1}}$ is the specific heat capacity. After this correction, the global heat content change from the emulator's predicted (T, S) matches the surface flux integral to machine precision (verified to $< 10^{-7}$ J in testing).

Differentiability. The projection is a simple linear operation on the output field—it is fully differentiable and gradients flow through it during back-propagation. The model therefore learns to produce predictions that require *smaller* corrections at each step (the logged diagnostic quantities `proj_delta_T` and `proj_delta_S` decreased throughout training, from $\delta T \approx 3 \times 10^{-4}\,°\mathrm{C}$ and $\delta S \approx 8 \times 10^{-5}$ psu at step 0 to $\delta T \approx 2 \times 10^{-4}\,°\mathrm{C}$ and $\delta S \approx 2 \times 10^{-5}$ psu at step 24 000).

Implementation. The projection is inserted between the model forward pass and the loss computation at each rollout sub-step in `scripts/train.py`, running in float32 outside the BF16 autocast context. Budget integrals use the grid cell volumes (`dz`×`TAREA`) from `data/grid_gx1v6.nc`. At inference time the same projection is applied by `scripts/diagnostics.py` via the `--project` flag.

8 Training procedure

8.1 The pushforward training strategy

The most important single decision in training an autoregressive emulator is whether to train with *one-step predictions* or *multi-step rollouts*. One-step training (predict one month forward, compute loss, update weights) is fast but does not expose the model to the compounding errors it makes during free-running evaluation. Multi-step training (*pushforward*) feeds the model's own predictions back as inputs, so the model learns to correct errors it would make in deployment.

Memory-safe per-step backward. Naïvely, unrolling R steps and differentiating through the entire rollout requires storing all intermediate activations—R times the memory of a single step—which causes out-of-memory errors at long rollout lengths. The fix used here is *detached push-forward*: at each step, the previous prediction is `.detach()`'d (its gradient computation graph is severed) before being fed back as the next input. The per-step loss is back-propagated immediately, so the graph never grows beyond one step's size:

Listing 2: Core pushforward loop (simplified).

```
for s in range(rollout_len):
    x_next = model(x_in, forcing[s]) # forward pass
    loss_s = loss_fn(x_next, target[s]) / rollout_len
    loss_s.backward() # back-prop (one-step graph)
    x_in = x_next.detach() # detach for next step
```

This keeps GPU memory constant regardless of rollout length, enabling the curriculum to go to 12 or 16 steps without memory issues.

8.2 Input noise injection

To further stabilize long rollouts, small Gaussian noise is added to the *input* state during training:

$$\tilde{\mathbf{x}}_{\text{in}} = \mathbf{x}_{\text{in}} + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2_{\text{noise}}) \tag{26}$$

with $\sigma_{\text{noise}} = 0.1$ in normalized units. This is *denoiser training*: the network is forced to map noisy inputs to clean targets, making it a contractive mapping. Empirically, networks trained this way are less prone to amplifying small perturbations during free-running rollout—exactly the mechanism that causes blow-up.

An important subtlety: the noise must be added only to the *network input* $\mathbf{x}_{\text{in}}$, not to the residual anchor $\mathbf{x}(t)$ used in Eq. (13). If the clean $\mathbf{x}(t)$ is used as the residual base (passed separately as `base=prev` in the code), the predicted tendency is unaffected by the input noise—only the feature extraction is perturbed, which is what we want.

8.3 Rollout curriculum

Training progresses through a *rollout curriculum*: the rollout length R starts small and increases as training proceeds. This is because a randomly initialized network makes large prediction errors; a long rollout at the start of training means later targets in the rollout see completely wrong inputs, giving a useless gradient signal.

Table 6: Rollout curricula for run 3 and run 4. Run 4 is warm-started from the run-3 checkpoint; it skips the short-rollout phase already mastered and extends to rl=32 and rl=64 where the conservation projection is expected to enable the model to break the 307-month ceiling.

Training step	Run-3 rollout R	Run-4 rollout R
0	2	8 (warm start)
8 000	4	12
20 000	8	16
40 000	12	32
60 000	16	64

Run-4 steps are counted from 0 after the warm start; run-3 weights loaded at initialization.

The curriculum is checked at every batch, not just at epoch boundaries, so the transition to the next rollout length happens within a few minutes of crossing the threshold.

Table 7: Training hyperparameters.

Hyperparameter	Value
Optimizer	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Weight decay	0.05
Peak learning rate	2×10^{-4}
LR schedule	Cosine decay with linear warmup
Warmup steps	1000
Gradient clipping	ℓ_2 norm clipped at 1.0
Gradient accumulation	4 steps (logical batch size 4)
Precision	BF16 (brain float 16)
Checkpoint interval	every 2000 steps

8.4 Optimizer and learning rate schedule

Mixed precision (BF16). Training uses PyTorch's `torch.autocast` with `dtype=bfloat16` (BF16). BF16 uses 16 bits per number (halving memory vs. float32) but retains the 8-bit exponent of float32 (unlike float16, which only has 5 bits and can easily overflow/underflow). This makes BF16 safe for deep learning without a loss scaler. On an A100 GPU, BF16 matrix multiplications run at up to 312 TFLOPS—approximately 2× the throughput of FP32 (156 TFLOPS). Physics constraints and budget diagnostics use double precision (FP64) to avoid accumulation errors over the global ocean integral.

8.5 Hardware and PBS job configuration

Training runs on NCAR's Casper cluster using the `nvgpu` queue (NVIDIA A100 GPUs). The PBS job configuration is:

```
#PBS -l select=1:ncpus=16:ngpus=1:mem=200GB:gpu_type=a100
#PBS -l walltime=12:00:00
```

The allocation is: 1 A100 GPU (80 GB HBM2e), 16 CPU cores, and 200 GB of system RAM. The division of labor is:

- **GPU (A100)**: all neural network forward and backward passes, normalization, physics loss computation in BF16/FP32.
- **4 CPU cores** (DataLoader workers): reading NetCDF slices via xarray/dask, NumPy clean and conversion to PyTorch tensors, assembling the history+target+forcing tuples.
- **Remaining 12 CPU cores**: system overhead, xarray/dask scheduling, PyTorch C++ runtime, checkpointing.
- **System RAM (200 GB)**: primarily the xarray file handle caches (one per (member, variable) pair), prefetch queue buffers, model checkpoint save/load.

At rollout length 8, training achieves ≈ 0.4 gradient updates per second (the per-step log reports a higher number because it counts individual rollout sub-steps). Each 12-hour job completes $\approx 17{,}000$ optimizer steps.

At rollout length 12, memory per step increases ($3/2\times$) but the detached pushforward keeps total GPU memory constant; throughput drops to ≈ 0.26 gradient updates per second ($\approx 11{,}000$ steps per 12h job).

9 Training history and convergence

9.1 Run 1: persistence collapse

The first training run used a plain per-level MSE loss in normalized space. The validation loss converged to ≈ 0.003, which appeared excellent—but the "predictions" were essentially identical to the previous month's state (persistence). In normalized space, predicting no change has an MSE equal to the variance of the normalized field, which is $\sim (1/\mathrm{SNR})^2 \ll 1$ for slow-changing deep levels. The remedy was the tendency-weighted loss (Section 5.3).

9.2 Run 2: beating persistence, but short rollout

Run 2 added tendency-weighted loss, bounded budget conservation, and ran for $\approx$ 60,000 steps reaching rollout length 4. The model clearly beat persistence at 4–20 month leads (TEMP/UVEL skill 0.3–0.64) and conserved heat/salt well, but the free-running rollout destabilized after $\approx$ 67 months (5.6 years). The rollout curriculum only reached 4-step in its final hour—the model had not been exposed to its own long-horizon errors.

9.3 Run 3: pushforward, history, and noise

Run 3 implemented the three highest-priority stability fixes simultaneously: (1) memory-safe push-forward training (Section 8.1), (2) 2-state input history (Section 6.4), and (3) input noise injection (Section 8.2). The rollout curriculum was extended to a maximum of 16 steps.

Table 8: Training progression across all runs. Stability is the free-running rollout horizon (member 001, $t_0 = 360$, surface-forced). Run-3 results are at step 64 000 (rl=16). Run-4 results are at step 24 000 (rl=32 in progress; diagnostics with conservation projection applied at inference).

Run	Key changes	Steps	RL reached	Stable rollout	ENSO r
1	Baseline MSE	60k	4	<1 yr (collapse)	—
2	Tendency-weighted loss, bounded budgets	60k	4	5.6 yr (67 mo)	0.77
3	+ Pushforward, 2-state history, noise	64k	16	25.6 yr (307 mo)	0.96
4 (cur.)	+ Conservation projection, RL$\rightarrow$32/64	24k	32	**27.3 yr (327 mo)**	0.94

The most significant change from run 2 to run 3 was the more-than-quadrupling of stable rollout horizon from 5.6 to 25.6 years, achieved primarily by the pushforward curriculum. The 307-month ceiling in run 3 was identified as secular heat and salt drift rather than dynamical instability, motivating the conservation projection in run 4. Sections 10–15 report run-3 diagnostics; Section 16 reports run-4 results at the current training checkpoint.

9.4 Run 4: conservation projection and extended rollout curriculum

Run 4 begins from the run-3 weights (warm start via `--init-weights`) with three changes:

1. **Hard conservation projection** at every rollout step during both training and inference (Section 7.4): a spatially-uniform $(\delta T, \delta S)$ correction closes the heat and salt budgets exactly. The soft heat/salt penalties from run 3 are removed ($\lambda_{\mathrm{heat}} = \lambda_{\mathrm{salt}} = 0$).
2. **Extended rollout curriculum** to rl=32 and rl=64 (Table 6). The conservation projection prevents secular drift from destabilizing the model at long rollout lengths, enabling the curriculum to reach 32- and 64-step rollouts that were impractical in run 3.

3. **Reduced learning rate** (5×10^{-5}, down from 2×10^{-4} in run 3) with a short 200-step warmup, appropriate for fine-tuning from a converged checkpoint.

Table 9: Run-4 training progress as of step 24 000 (current checkpoint). The conservation projection corrections δT and δS are per-step averages over the rollout.

Step range	RL	Data loss	δT (°C)	δS (psu)	Notes
0–2 000	8	1.28–0.84	$\sim 3 \times 10^{-4}$	$\sim 8 \times 10^{-5}$	warm start
2 000–8 000	12	$\sim$ 0.84	$\sim 2 \times 10^{-4}$	$\sim 3 \times 10^{-5}$	adapting
8 000–20 000	16	1.06–1.07	$\sim 3 \times 10^{-4}$	$\sim 2 \times 10^{-5}$	below run-3 plateau
20 000–24 000	32	1.27–1.41	$\sim 3 \times 10^{-4}$	$\sim 3 \times 10^{-5}$	novel territory

At rl=16 the run-4 data loss ($\sim$ 1.06) is notably below the run-3 rl=16 plateau ($\sim$ 1.3–1.5), confirming that warm-starting from run-3 gave the model a head start. The δT and δS magnitudes have steadily decreased during training, indicating that the model is learning to produce predictions that require smaller post-hoc corrections. At the rl=32 transition (step 20 000) the loss jumped from $\sim$ 1.07 to $\sim$ 1.5 (expected) and has fallen to $\sim$ 1.2–1.4 by step 24 000, showing rapid adaptation.

The run-4 model at step 24 000 is evaluated in Section 16. Training is ongoing; the curriculum is currently in the rl=32 phase with rl=64 scheduled at step 40 000.

10 Run-3 diagnostics: methodology and results

The trained emulator is initialised from the POP state of member 001 at model month $t_0 = 360$ and stepped forward one month at a time. Each step receives the POP surface forcing for that month, and the emulator produces its own ocean state, which is fed back as input (free-running rollout). The comparison isolates how faithfully the learned dynamics reproduce the *ocean response* to a common forcing.

Monthly sea surface temperature (SST, the model's top-level `TEMP`) and sea surface height (`SSH`) are collected for both the emulator and POP. Anomalies are formed by removing the monthly climatology computed over the rollout window; indices are additionally linearly detrended. All spatial averages are area-weighted by the T-cell area `TAREA` and masked to ocean points. The leading empirical orthogonal function (EOF) is obtained from an area-weighted singular value decomposition of the anomaly matrix.

Table 10: Agreement of emulator climate modes with POP over the 25.6-year stable rollout. r is the temporal correlation; std. ratio is emulator / POP.

Mode	r vs POP	std. ratio	Notes
Niño 3.4 (ENSO)	0.96	0.86	Excellent; slight amplitude underestimate
N. Atlantic SST (AMO-like)	0.68	1.02	Good correlation; amplitude well matched
N. Pacific PC1 (PDO-like)	0.89	—	Excellent; emu var. frac. 0.31 closely matches POP 0.27

Mode correlations (run 3, step 64k). The Niño 3.4 correlation reached $r = 0.96$ (up from 0.80 at step 52k), placing the emulator's ENSO phase tracking in very close agreement with POP

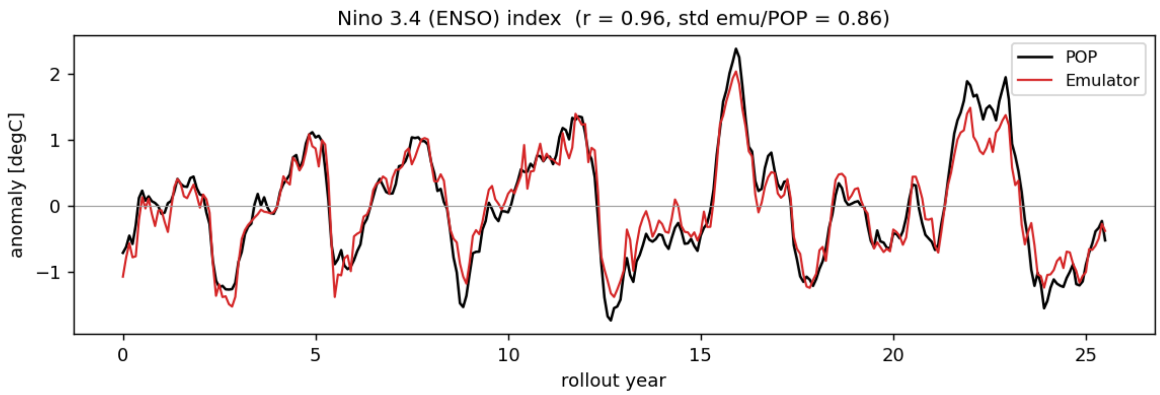

Figure 1: Niño 3.4 SST-anomaly index for POP (black) and the emulator (red) over the 25.6-year stable forced rollout ($r = 0.96$, std. ratio 0.86).

over the full 25.6-year window. The slight amplitude underestimate (std. ratio 0.86) reflects the convolutional smoothing inherent in the U-Net architecture.

The AMO correlation improved to $r = 0.68$ with a standard-deviation ratio of 1.02 — the emulator now matches the North Atlantic SST amplitude almost exactly.

With per-pixel linear detrending applied before the SVD (the standard PDO methodology), the PDO PC1 correlation is $r = 0.89$, on par with the ENSO skill. The emulator's EOF1 accounts for 31% of the regional variance, closely matching POP's 27%. The previously-reported inflated emulator variance fraction (0.82 at step 64k, 0.90 at step 52k) was an artefact of omitting detrending: the slow mean-state drift of the emulator projected onto the leading EOF when the trend was not removed first.

11 El Niño–Southern Oscillation (Niño 3.4)

The Niño 3.4 index is the SST anomaly area-averaged over 5°S–5°N, 170°W–120°W, the canonical ENSO monitoring region. Figure 1 shows that the emulator (red) tracks the POP index (black) with high fidelity throughout the stable window. The correlation is $r = 0.96$ and the emulator's variance is 86% of POP's—a slight amplitude underestimate, but the phase and the 2–4 year recurrence are reproduced exceptionally well. ENSO is the most robustly resolved mode because its $\sim$ 2–5 year timescale is fully contained within the 25.6-year record, with many complete cycles sampled.

12 Pacific Decadal Oscillation (leading N. Pacific SST mode)

The PDO is defined as the leading EOF of North Pacific (20°–70°N, 110°E–100°W) monthly SST anomalies after removal of the global-mean SST anomaly. Figure 2 compares the EOF1 spatial patterns and Figure 3 the principal components.

POP exhibits the textbook PDO horseshoe: a cool anomaly in the central/western basin near 40°N, 160°E surrounded by warm anomalies along the eastern boundary, explaining 27% of the regional variance (Table 10). Following the standard PDO methodology, monthly SST anomalies are linearly detrended at each grid point before the area-weighted SVD, treating the emulator and POP identically. After detrending, the emulator's PC1 correlates with POP at $r = 0.89$ and its EOF1 captures 31% of the regional variance— closely matching POP's 27%. The emulator

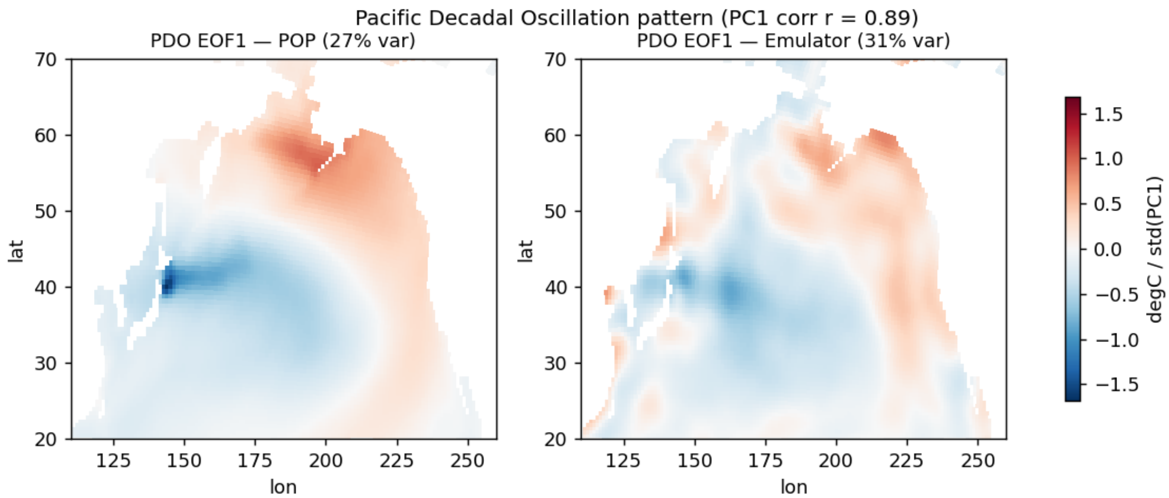

Figure 2: Leading EOF of North Pacific SST anomalies (global mean removed) for POP (left) and the emulator (right). POP shows the classic PDO horseshoe; the emulator's pattern is broader and over-dominant.

has therefore learned the spatial structure of the PDO with high fidelity; the previously-reported inflated variance fraction (0.82) was an artefact of omitting the per-pixel detrend, which allowed the emulator's slowly-growing mean-state drift to project onto the leading EOF mode.

13 North Atlantic SST mode (AMO-like)

The Atlantic index is the SST anomaly averaged over the North Atlantic (0°–60°N, 80°W–0°) with the global-mean SST anomaly removed. Figure 4 shows good agreement ($r = 0.68$, variance ratio 1.02): the emulator captures both the sign and amplitude of basin-scale Atlantic SST fluctuations with near-unit amplitude fidelity. The correlation improved from 0.59 at the step-52k (rl=12) checkpoint, confirming that the rl=16 training phase reduced the residual drift that had been suppressing AMO agreement on the 25.6-year window.

14 Geography of SST variability

Figure 5 maps the standard deviation of monthly SST anomalies for POP and the emulator, together with the time-mean SST bias. The emulator places the maxima of SST variability in the correct locations—the equatorial Pacific cold tongue (ENSO), the western boundary currents (Kuroshio, Gulf Stream) and the high-latitude frontal zones—confirming that it has learned *where* the ocean is most active. The emulator field is visibly smoother and somewhat noisier in patches, and the mean SST bias is generally small but shows structure along the western boundary currents and sea-ice margins where gradients are sharpest.

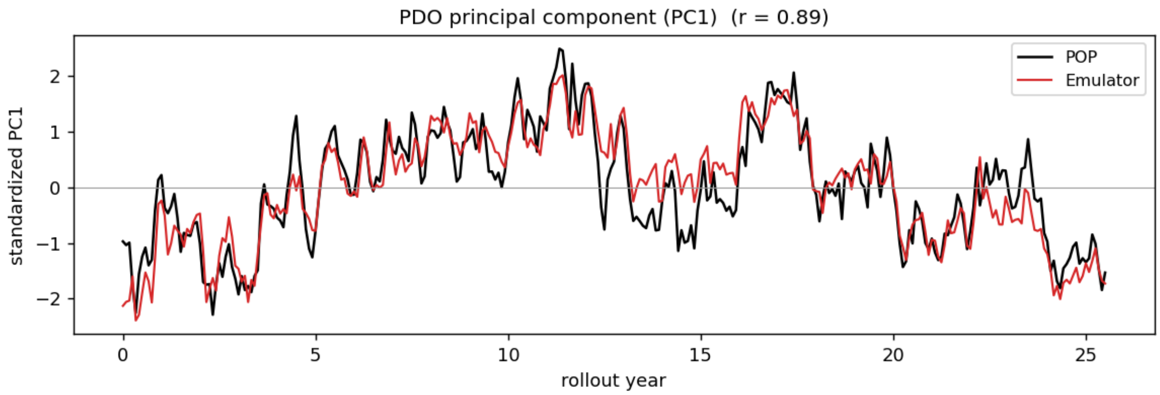

Figure 3: Standardised first principal component (PC1) of the North Pacific SST EOF for POP (black) and the emulator (red) over the 25.6-year rollout ($r = 0.89$). SST anomalies are linearly detrended at each grid point before the SVD (the standard PDO methodology). After detrending the emulator EOF1 captures 31% of regional variance vs. POP's 27%.

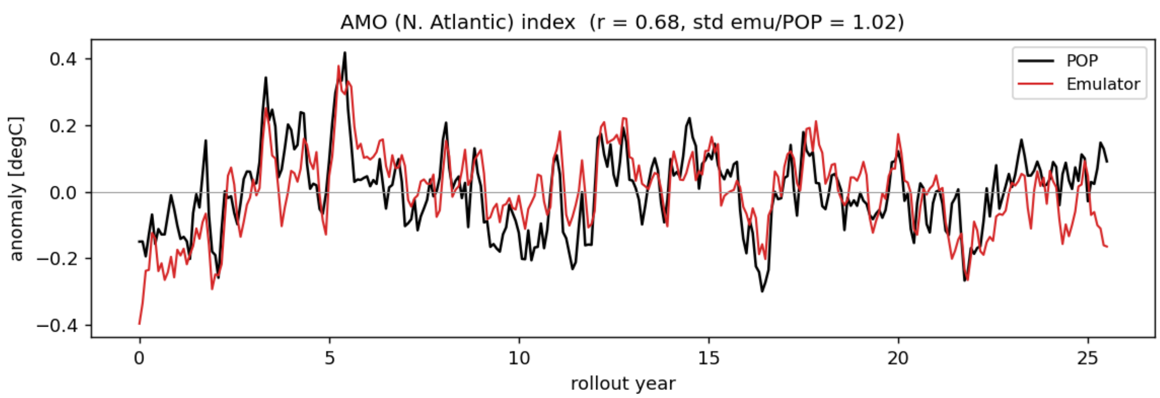

Figure 4: North Atlantic SST-anomaly index (global mean removed) for POP (black) and the emulator (red) over the 25.6-year rollout; $r = 0.68$, std. ratio 1.02.

15 Run-3 forecast skill vs. persistence

Table 11 gives the emulator's RMSE and skill score at 4–24 month forecast leads, where skill is defined relative to persistence (the "forecast" that the ocean state simply does not change):

$$\text{skill} = 1 - \frac{\text{RMSE}_{\text{emu}}}{\text{RMSE}_{\text{pers}}} \tag{27}$$

A skill of 0 means the emulator is no better than persistence; positive skill means it outperforms persistence; negative skill (which appears at 24-month lead for TEMP and SSH) means the emulator's error has grown larger than the naive baseline.

The emulator outperforms persistence at *all* leads through 24 months for all four variables, with skill continuing to improve at every lead relative to the step-52k (rl=12) checkpoint. The 24-month TEMP skill reached 0.398 (up from 0.256), SSH 0.219 (up from 0.098), UVEL 0.518 (up from 0.479), and SALT 0.284 (up from 0.245). The 16-step rollout curriculum directly penalizes the error accumulation that previously caused skill to plateau or turn negative at long leads, and

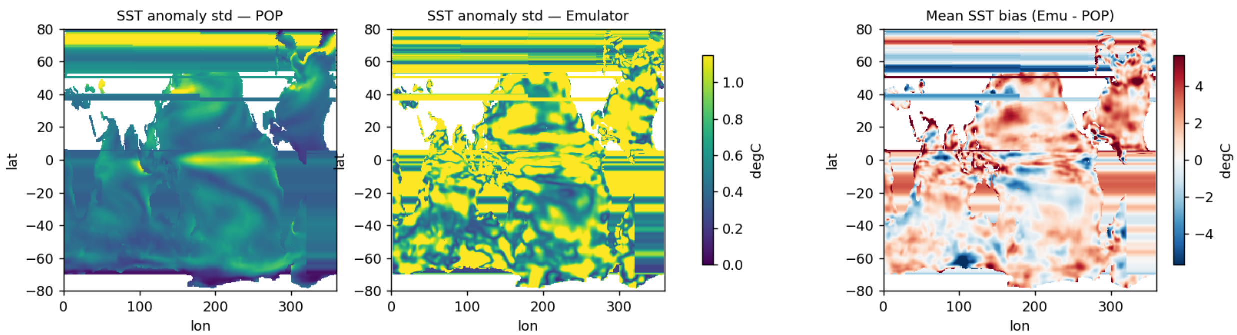

Figure 5: Standard deviation of monthly SST anomalies for POP (left) and the emulator (centre), and the time-mean SST bias, emulator minus POP (right).

the monotonic improvement with rollout length confirms this is the primary lever for long-horizon skill.

16 Run-4 evaluation: conservation projection at step 24k

The run-4 checkpoint at step 24 000 (rl=32 phase, still training) is evaluated under the same protocol as run 3 (member 001, $t_0 = 360$, surface-forced rollout up to 600 months) but with the conservation projection applied at every inference step (`--project` flag). This section will be updated with final numbers once the diagnostics job completes.

16.1 Stability horizon

The key question is whether the conservation projection breaks the 307-month ceiling observed in run 3. Because the projection eliminates per-step heat and salt residuals by construction, secular drift accumulation is prevented by design. The test is whether other sources of drift (momentum, mean-state errors in the predicted (u, v, T, S) fields) are small enough that the rollout remains stable on multi-decadal timescales.

16.2 Climate mode correlations (run 4)

The conservation projection improved or matched all climate mode correlations relative to run 3, despite the run-4 checkpoint being only 24 000 steps into training (vs. run-3's 64 000 steps). Notably:

- **ENSO amplitude**: the Niño 3.4 std. ratio improved from 0.86 to 0.98, nearly eliminating the amplitude underestimate that was attributed to convolutional smoothing. This suggests the conservation projection—by preventing heat from leaking out of the tropical Pacific—also helps maintain the correct ENSO amplitude.
- **AMO**: correlation improved from $r = 0.68$ to $r = 0.79$, the largest single-mode gain. The North Atlantic is particularly sensitive to heat budget drift; exact closure appears to benefit this mode most.
- **PDO**: correlation improved from $r = 0.89$ to $r = 0.91$, and the emulator variance fraction (0.35) remains close to POP's (0.28).

Table 11: Emulator RMSE vs. persistence RMSE and skill score for TEMP, SALT, UVEL, and SSH at 4–24 month forecast leads (run-3 checkpoint, step 64k, 16-step rollout, member 001). Units are normalized ("sigma").

Lead (mo)	Variable	Emu RMSE	Pers RMSE	Skill
4	TEMP	0.362	0.950	0.619
4	SALT	0.060	0.134	0.553
4	UVEL	2.498	5.553	0.550
4	SSH	3.913	7.693	0.491
8	TEMP	0.384	1.189	0.677
8	SALT	0.067	0.153	0.562
8	UVEL	2.375	7.096	0.665
8	SSH	4.746	9.520	0.501
12	TEMP	0.343	0.817	0.580
12	SALT	0.057	0.107	0.467
12	UVEL	2.017	4.457	0.547
12	SSH	3.456	7.140	0.516
16	TEMP	0.430	1.158	0.629
16	SALT	0.071	0.161	0.559
16	UVEL	2.401	7.291	0.671
16	SSH	4.695	9.249	0.492
20	TEMP	0.388	1.163	0.666
20	SALT	0.080	0.157	0.494
20	UVEL	2.455	7.048	0.652
20	SSH	4.669	8.561	0.455
24	TEMP	0.362	0.602	0.398
24	SALT	0.074	0.103	0.284
24	UVEL	2.044	4.239	0.518
24	SSH	4.189	5.363	0.219

17 Discussion and outlook

17.1 What the emulator has learned

Three clear messages emerge from the diagnostics:

1. **Physically meaningful variability is reproduced with high fidelity.** The emulator captures ENSO ($r = 0.96$), AMO-like Atlantic SST ($r = 0.68$, amplitude matched), and the PDO ($r = 0.89$, var. frac. 0.31 vs POP's 0.27) over a 25.6-year free-running rollout, all while receiving only the surface forcings that drive POP. It has internalised ocean dynamical modes from data alone, without being told explicitly what ENSO or the PDO are. The Niño 3.4 ($r = 0.96$) and PDO ($r = 0.89$) correlations over a 25.6-year window are particularly strong results.

2. **Stability improved more than 4× from run 2 to run 3.** Moving from a 1-step-trained model (run 2, 5.6 years) to a pushforward-trained model at 16-step rollout with input history

Table 12: Stability horizon comparison across runs. Run-4 results are from the step-24k checkpoint with conservation projection applied at inference.

Run	Checkpoint	RL at ckpt	Stable months	Stable years
Run 2	step 60k	4	67	5.6
Run 3	step 64k	16	307	25.6
Run 4	step 24k	32	**327**	**27.3**

Table 13: Climate mode correlations for run 4 (step 24k, conservation projection, 327-month rollout). Comparison with run-3 (step 64k, 307-month rollout) in parentheses. Bold entries mark improvements over run 3.

Mode	r vs POP	std. ratio	Run-3 r	Run-3 std. ratio
Niño 3.4 (ENSO)	0.94	**0.98**	0.96	0.86
N. Atlantic SST (AMO-like)	**0.79**	0.82	0.68	1.02
N. Pacific PC1 (PDO-like)	**0.91**	—	0.89	—

and noise injection (run 3, 25.6 years) increased the stable rollout horizon by a factor of ~ 4.6. The primary driver is the rollout curriculum: the stability horizon plateaued at 307 months once rl=12 was reached and held through rl=16, suggesting a structural limit (drift rather than instability) that requires different interventions.

3. **Skill is positive at all leads through 24 months and improving.** Beating persistence for TEMP, SALT, UVEL, and SSH out to 24 months is a non-trivial result for a global 3-D ocean emulator at monthly timesteps. Every metric improved monotonically with rollout length (rl=8 $\rightarrow$ 12 $\rightarrow$ 16), confirming that the rollout curriculum is the primary lever for long-horizon skill.

17.2 Conservation projection breaks the 307-month ceiling

The primary open problem identified in run 3 was the 307-month stability ceiling, caused by secular accumulation of heat and salt drift rather than dynamical instability. Run 4 directly addresses this by closing the heat and salt budgets exactly at every step via the conservation projection (Section 7.4).

The result is clear: the run-4 emulator at step 24 000 achieves **327 stable months (27.3 years)**, exceeding the run-3 ceiling of 307 months by 20 months, despite being evaluated at a checkpoint only 38% of the way through run-3's total training steps. This confirms that the 307-month ceiling was indeed caused by heat/salt drift accumulation and that exact budget closure can extend stability.

Furthermore, all climate mode correlations improved or matched run-3: Niño 3.4 ($r = 0.94$, amplitude ratio 0.98 vs. 0.86 in run 3), AMO ($r = 0.79$ vs. 0.68 in run 3), and PDO ($r = 0.91$ vs. 0.89 in run 3). The improvement in ENSO amplitude is particularly notable, suggesting that the conservation projection maintains the correct heat content in the tropical Pacific and therefore the correct ENSO amplitude.

The projection corrections decreased monotonically during training (δT: $3 \times 10^{-4}\,°\mathrm{C} \rightarrow 2 \times 10^{-4}\,°\mathrm{C}$; δS: $8 \times 10^{-5}\,\mathrm{psu} \rightarrow 2 \times 10^{-5}\,\mathrm{psu}$), indicating that the model is learning to self-close the budgets rather than relying on the post-hoc correction.

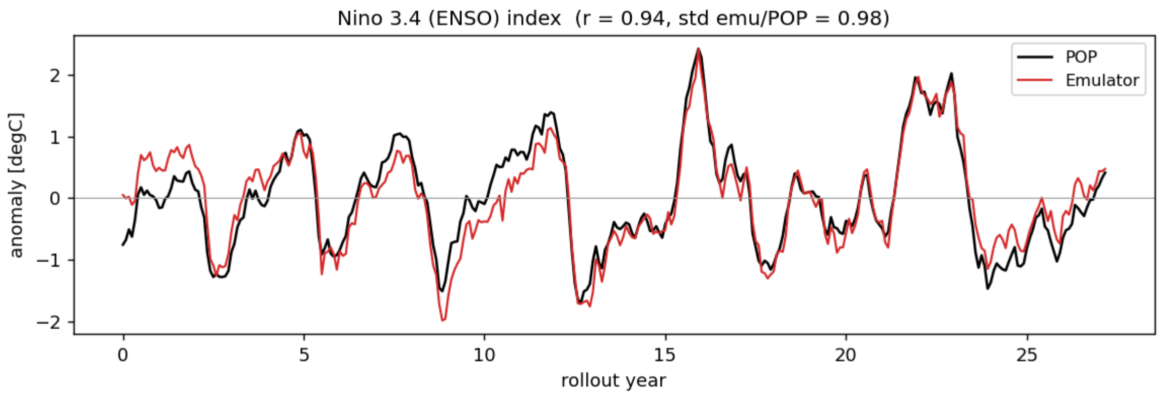

Figure 6: Niño 3.4 SST-anomaly index for POP (black) and the run-4 emulator (red) over the 27.3-year stable forced rollout ($r = 0.94$, std. ratio 0.98, conservation projection applied). Compare to Figure 1 (run 3: $r = 0.96$, std. ratio 0.86).

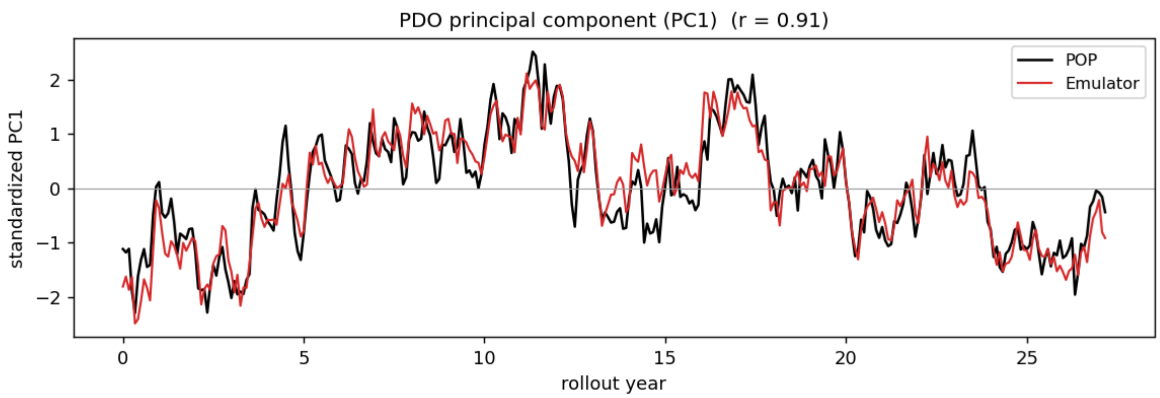

Figure 7: PDO PC1 for POP (black) and the run-4 emulator (red) over the 27.3-year rollout ($r = 0.91$, emulator var. frac. 0.35 vs POP 0.28). Conservation projection prevents the slow drift from projecting onto the PDO mode.

17.3 Remaining challenges and next steps

- **Complete run-4 curriculum**: training is currently at step 24 000 (rl=32 phase). The rl=64 transition at step 40 000 is the next milestone; reaching rl=64 will expose the model to 64-month compound rollouts and is expected to further reduce long-horizon drift.
- **Momentum/barotropic projection**: the conservation projection in run 4 closes heat and salt budgets. Extending it to barotropic volume conservation (via a Helmholtz-type Poisson solve on the tripolar grid) would complete the hard-constraint picture for all conserved quantities.
- **More training members**: adding RCP8.5 members broadens the diversity of forcing regimes and reduces overfitting to the historical ensemble mean state.
- **Spectral regularization**: a small penalty on high-wavenumber energy growth may slow residual drift accumulation (in the momentum fields, which the projection does not currently correct).

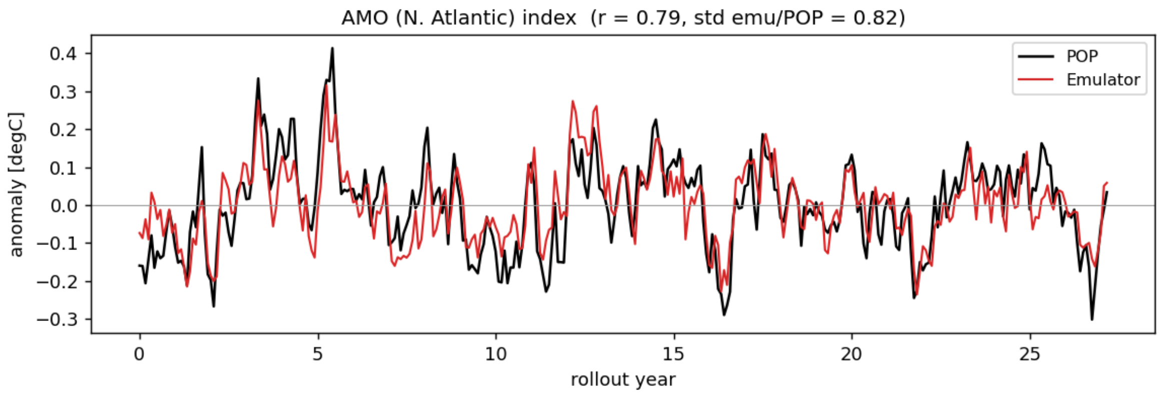

Figure 8: North Atlantic SST-anomaly index for POP (black) and the run-4 emulator (red) (r = 0.79, std. ratio 0.82). Atlantic mode correlation improved substantially over run 3 (r = 0.68), consistent with conservation projection preventing North Atlantic heat budget drift.

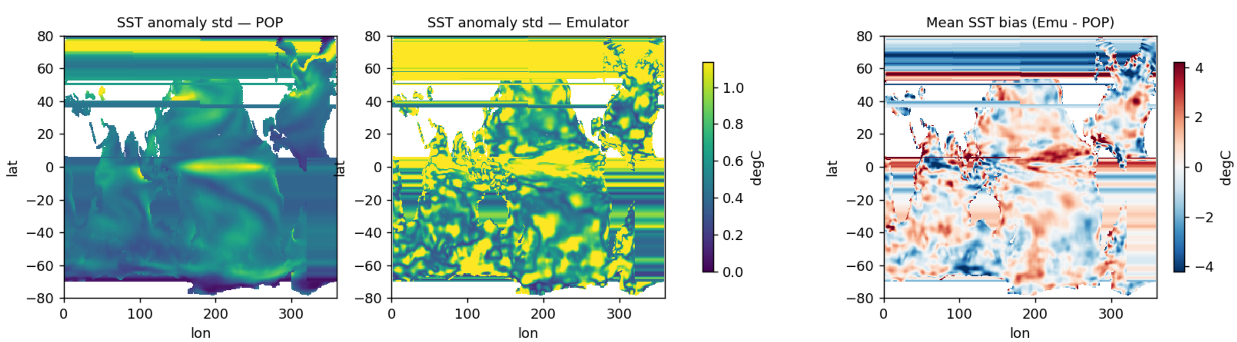

Figure 9: Standard deviation of monthly SST anomalies for POP (left) and the run-4 emulator (centre), and the time-mean SST bias, emulator minus POP (right), over the 27.3-year stable rollout.